

Relatório Final: Arquitetura de Software para Monitoramento e Prevenção de Incêndios Florestais Baseada em IoT, Apache Kafka e Spark Structured Streaming

1. Contexto e Motivação

O aumento na frequência e na severidade dos incêndios florestais impõe a necessidade de sistemas de monitoramento ambiental baseados em soluções de tempo real e alta escalabilidade. Abordagens tradicionais, como o sensoriamento remoto por satélites ou o patrulhamento visual humano, falham em prover a granularidade temporal necessária para a supressão de focos de incêndio em estágios iniciais, devido a limitações de órbita, cobertura de nuvens ou atrasos na transmissão de alertas. Este projeto visa resolver tais gargalos por meio do desenvolvimento de uma arquitetura *end-to-end* voltada à detecção precoce de anomalias climáticas.

2. Dados

2.1 Descrição Detalhada

- **Origem:** Dados gerados sinteticamente pelo script `src/simulador_stress2.py`.
- **Conteúdo:** Mensagens estruturadas em formato JSON, contendo identificador único (`sensor_id`), marcação temporal (`timestamp`) e variáveis físicas ambientais (temperatura, umidade e CO2).
- **Amostra:** Um arquivo de amostra compacto (`sample_sensor.json`) é fornecido na pasta `datasample/` para inspeção rápida.

2.2 Como obter os dados

Não é necessário o download de conjuntos de dados externos. O sistema utiliza um simulador estocástico que gera o fluxo de dados em tempo real, orquestrado via Docker. O código-fonte está disponível em: https://github.com/pclinux-48/topics_computer_systems.

3. Como Instalar e Executar

Início Rápido: Navegue até a pasta `gx/` e execute `./bin/iniciar_tudo.sh`. Este script inicializa os contêineres, configura o ambiente virtual Python e inicia o pipeline.

Monitoramento: Após a inicialização, a interface web está acessível em `http://localhost:8501`.

Observação: Utilize `./bin/observar_pipeline.sh` para visualizar os logs de Kafka, Spark e PostgreSQL em tempo real.

4. Arquitetura do Projeto

O sistema adota um paradigma de sistemas distribuídos fracamente acoplados.

- **Componentes:**

Captação: Nós simulados (Edge AI) processam condições locais.

Ingestão: Apache Kafka atua como *buffer* resiliente.

Processamento: Spark Structured Streaming computa o Índice de Risco de Incêndio (IRI).

Data Sink: PostgreSQL (JSONB) para armazenamento interoperável.

5. Cargas de Trabalho Avaliadas

- **[WORKLOAD-1] Processamento em Fluxo:** Consumo e agregação analítica no Spark, aplicando regras de curto-circuito lógico.
- **[WORKLOAD-2] Persistência em Banco de Dados:** Ingestão via JDBC em colunas JSONB, testada sob estresse de até 5.000 nós concorrentes.

6. Experimentos e Resultados

6.1 Ambiente Experimental

Os experimentos foram executados em ambiente orquestrado via Docker, submetendo o broker Apache Kafka a cargas progressivas.

6.2 Análise de Resultados

A arquitetura demonstrou notável estabilidade até o limiar de 1.000 nós.

Quantidade de Sensores	Latência Média (ms)	Taxa de Transmissão (msg/s)
------------------------	---------------------	-----------------------------

20	4,6	4,0
1.000	39,0	200,0
5.000	607,2	1000,0

Discussão: O sistema absorveu fluxos contínuos de maneira linear e estável até 1.000 dispositivos. Em cargas extremas de 5.000 nós, observou-se um aumento exponencial na latência, mapeando o limiar de escalabilidade da infraestrutura.

7. Limitações e Conclusões

O trabalho validou com êxito uma arquitetura de alta resiliência e baixo custo para monitoramento ambiental. Como limitação, identificou-se que, acima de 400 nós concorrentes, o gargalo de I/O JDBC no PostgreSQL indica a necessidade de evolução para sistemas de arquivos distribuídos em escalas macro-ambientais.